

Class 13: Data Types and Structures for Data-Driven Systems

Data-driven Systems Engineering

Agenda

- Why identifying data type comes before any modeling decision.
- Tabular data: rows, columns, and the independence assumption.
- Time series data: order, timestamps, and temporal dependency.
- Spectral and signal data: waveforms and frequency-domain thinking.
- Image data: pixels, grids, and spatial structure.
- How the fraud and pasteurization cases embody different data types.

Learning goals

- Recognize that “a dataset” is not one thing: tabular, time series, spectral, and image data carry different assumptions.
- Identify which assumptions (independence, ordering, locality) each data type licenses or forbids.
- Connect each data type to a concrete artifact already in this repository.
- Explain why data type must be settled before encoding, feature engineering, or model choice.

Course Map and Running Cases



Today in the course

Focus: Recognizing tabular, time series, spectral, and image data before any modeling begins.

Bridge: This class opens the Data-Driven block: after twelve classes of Python fluency, we now slow down and ask what kind of data we actually have, before choosing how to represent or model it.

Fraud case

The fraud case is tabular: one row per transaction, columns of named or anonymized attributes, and no assumed order between rows.

Pasteurization case

The pasteurization case is a time series and signal problem: sensors sampled once per second, with heating and cooling dynamics that only make sense in temporal order.

Course thread: Every later class (encoding, feature engineering, modeling, serving) depends on having correctly

Why data type matters before modeling

- Algorithms encode implicit assumptions about structure: independence between rows, smoothness over time, or locality over space.
- Using a tabular technique on ordered data can silently destroy the information that matters most (the order itself).
- The same physical quantity, temperature, is tabular in one dataset and a time series in another, depending on how it was collected.
- Choosing the right data type framing narrows down encoding, feature engineering, and model family choices dramatically.

Tabular data

- Organized as rows (observations) and columns (variables), typically stored as a table, CSV file, or dataframe.
- Rows are usually assumed independent and identically distributed (i.i.d.): shuffling rows should not change what a model can learn.
- Columns can be numerical, categorical, or boolean, and each has a fixed meaning across all rows.
- This is the default assumption behind most classical machine learning tooling.

Tabular example: the fraud dataset

```
rename_map = {
    'Time': 'transaction_time',
    'Amount': 'transaction_amount',
    'Class': 'is_fraud'
}
for i in range(1, 29):
    rename_map[f'V{i}'] = f'V{i}_feature'
df.rename(columns=rename_map, inplace=True)
```

- This is from 'Credit_Card_Fraud_Detection_Solution.ipynb': one row per transaction, one named column per attribute.
- Nothing in the fraud table depends on which row came before it; 'transaction_time' is a column value, not a row ordering.

Four data types at a glance

Data type	Defining structure
Tabular	Independent rows, fixed columns; order between rows carries no meaning.
Time series	Ordered samples over time; past values constrain and explain future ones.
Spectral / signal	A waveform best understood in the frequency domain, not only the time domain.
Image	A 2D (or 3D) grid of values where neighboring pixels are strongly correlated.

Time series data

- Samples are ordered by timestamp, and that order is part of the data, not an artifact of storage.
- Neighboring points are correlated: today's sensor reading depends on yesterday's process state.
- Trends, seasonality, and regime changes (state transitions) are first-class phenomena.
- Splitting or shuffling a time series naively for training and testing leaks future information into the past.

Time series example: a pasteurization batch

```
def simulate_batch(bid, start_time=0.0):  
    pdur = {k: int(rng.integers(*PROD_RANGES[k])) for k in PROD_RANGES}  
    ...  
    frames = simulate_production_cycle(pdur, ctx)  
    df, t = to_df(frames, start_time, bid)  
    df["dTdt"] = df["T"].diff().fillna(0.0) / DT  
    return df, t
```

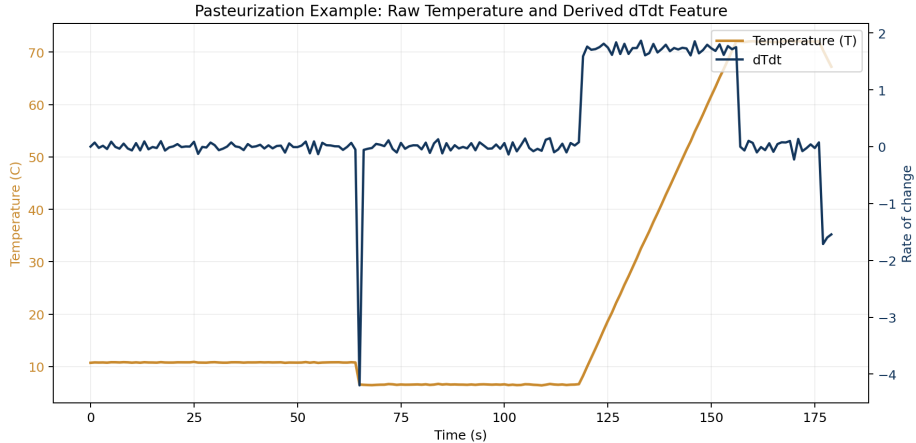
- 'pasteurization/synth_sensors.py' generates one row per second, per sensor, for an entire batch lifecycle.
- 'dTdt' only makes sense because rows are ordered: it is a derivative with respect to time, computed with '.diff()'.

A time series is often also a state machine



- These are exactly the six states used by 'PROD_RANGES' in 'synth_sensors.py'.
- Knowing the current state changes what a “normal” sensor reading looks like at that moment.

Stream example: from raw signal to derived feature



In 'pasteurization/synth_sensors.py', 'dTdt' is derived from the temperature time series to expose heating and cooling behavior directly.

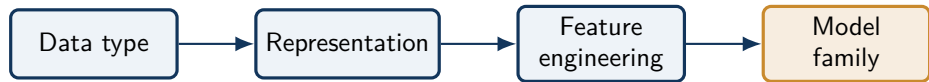
- A signal is a time series sampled densely enough that its frequency content becomes meaningful: audio, vibration, or high-rate sensor traces.
- The Fourier transform (FFT) re-expresses a signal as a sum of frequencies instead of a sequence of amplitudes over time.
- Audio pipelines commonly frame the signal into short overlapping windows and summarize each window (for example with MFCC features) before any modeling.
- This repository has no dedicated audio dataset; we reason about it conceptually and connect it to the pasteurization temperature trace, which is itself a densely sampled physical signal.

- An image is a 2D (or 3D, with color channels) grid of pixel intensities, not a table of independent variables.
- Spatial locality matters: a pixel is strongly related to its immediate neighbors, which is why convolution-based methods exist.
- There is no natural “row order” the way there is in tabular or time series data; the structure is spatial, not sequential.
- This repository has no image dataset either; it is included here so the taxonomy is complete before we discuss encoding and representation next class.

What breaks when the data type is ignored

Data type	What a naive tabular treatment breaks
Time series	Random train/test splits leak future information into the past.
Spectral / signal	Raw amplitude values hide the periodic structure that actually matters.
Image	Flattening pixels into columns destroys spatial neighborhoods.

From data type to system design



- Every arrow in this chain is a later class: representation and encoding (Class 14), feature engineering (Class 15), and modeling (Class 16–17).
- Getting the first box wrong quietly biases everything downstream.

Repo activity

- 'Credit_Card_Fraud_Detection_Solution.ipynb' and 'streamlit/modeling.py' are the tabular example: one row per transaction.
- 'pasteurization/synth_sensors.py' is the time series and signal example: seven sensors sampled once per second across a batch lifecycle.
- 'pasteurization/serving.py' turns that same time series into a live stream, which is the practical lab in Class 19.

Common mistakes when data type is ignored

- Treating a time-ordered sensor log as an i.i.d. table and shuffling it before splitting.
- Applying tabular feature-importance tools to raw pixel columns and expecting them to mean anything.
- Ignoring sampling rate: a signal sampled too slowly cannot represent the frequencies you care about.
- Forgetting that “more data” is not the same as “more independent observations” in time series and signals.

Question

The pasteurization sensors are logged into a table with one row per second. Is that table tabular data, time series data, or both?

- The goal is to separate storage format (a table) from data semantics (temporal dependency).
- This sets up the encoding and representation discussion in Class 14.

Papers and materials

Tukey (1977), Exploratory Data Analysis; Bishop, Pattern Recognition and Machine Learning; Chip Huyen, Designing Machine Learning Systems; pandas overview.

From This Class to the Next

What stays with us

- Data has an intrinsic type: tabular, time series, spectral/signal, or image, each with different structural assumptions.
- The fraud case is our tabular anchor; the pasteurization case is our time series and signal anchor.
- Getting the data type right is a precondition for every later step, not an optional first slide.

Project use

Before writing any preprocessing code for a project, teams should state explicitly which data type each of their inputs is.

Next connection

Class 14 asks how each data type is organized (structured, semi-structured, unstructured) and how we turn it into numbers through encoding and dimensionality reduction.

Course thread: Understanding data type is the first of four steps: type, representation, features, model.